

PAYMENT SERVICE - Información Completa

Conexión y URLs

- **URL Base:** <http://localhost:8002>
- **Puerto:** 8002
- **Ambiente:** development/production
- **Health Check:** GET /api/health

APIs Disponibles

Endpoints con Autenticación (JWT requerido):

POST /api/pagos # Crear nuevo pago
GET /api/pagos/:id # Verificar estado de pago por curso
GET /api/pagos/user/history # Historial de pagos del usuario
PUT /api/pagos/:id/status # Actualizar estado de pago (admin)

Webhooks (Sin autenticación):

POST /api/pagos/webhook # Webhook genérico
POST /api/pagos/paypal/webhook # Webhook PayPal específico
POST /api/pagos/coinbase/webhook # Webhook Coinbase específico
GET /api/pagos/paypal/callback # Callback PayPal (redirección)

Públicos:

GET /api/health # Health check del servicio
GET / # Info básica del servicio

Estructura de Datos

Respuesta Estándar:

json

```
{
  "success": true,
  "data": {
    "id": 123,
    "usuario_id": 456,
    "curso_id": 789,
    "estado": "aprobado",
    "monto": 99.99,
    "metodo": "paypal",
    "moneda": "USD",
    "transaccion_id": "pp_1234567890",
    "created_at": "2024-01-15T10:30:00Z",
    "updated_at": "2024-01-15T10:35:00Z"
  }
}
```

Estados de Pago:

- "pendiente" - Pago creado, esperando confirmación
- "aprobado" - Pago completado exitosamente
- "rechazado" - Pago rechazado por pasarela
- "cancelado" - Pago cancelado por usuario
- "reembolsado" - Pago reembolsado

Métodos de Pago:

- "paypal" - PayPal (integración real)
- "coinbase" - Coinbase Commerce (integración real)
- "stripe" - Stripe (simulado)
- "mercadopago" - MercadoPago (simulado)
- "tarjeta" - Tarjeta de crédito (simulado)
- "transferencia" - Transferencia bancaria (simulado)
- "dev" - Modo desarrollo (auto-aprueba)

Base de Datos

Tabla principal - pagos:

sql

```
CREATE TABLE pagos (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  usuario_id INT NOT NULL,  
  curso_id INT NOT NULL,  
  monto DECIMAL(10,2) NOT NULL,  
  metodo VARCHAR(50) NOT NULL,  
  estado VARCHAR(20) DEFAULT 'pendiente',  
  transaccion_id VARCHAR(100),  
  moneda VARCHAR(10) DEFAULT 'USD',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  deleted_at TIMESTAMP NULL,  
  INDEX idx_usuario_curso (usuario_id, curso_id),  
  INDEX idx_estado (estado),  
  INDEX idx_transaccion (transaccion_id)  
);
```

Tabla secundaria - transactions:

sql

```
CREATE TABLE transactions (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  payment_id INT NOT NULL,  
  provider VARCHAR(50) NOT NULL,  
  external_id VARCHAR(100) NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  amount DECIMAL(10,2) NOT NULL,  
  currency VARCHAR(10) NOT NULL,  
  raw_response TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (payment_id) REFERENCES pagos(id) ON DELETE CASCADE  
);
```

Autenticación

JWT Token requerido en headers:

```
Authorization: Bearer <jwt_token>
```

Claims del JWT:

json

```
{
  "user_id": 123,
  "role": "user",
  "exp": 1642686400,
  "iat": 1642600000
}
```

Tecnologías Stack

- **Go:** 1.21+
- **Framework:** Gin
- **ORM:** GORM
- **Base de datos:** MySQL/MariaDB
- **Autenticación:** JWT
- **Container:** Docker
- **PayPal SDK:** github.com/plutov/paypal/v4

Ejemplos de Uso

Verificar si usuario pagó un curso:

bash

```
curl -H "Authorization: Bearer <token>" \
  http://localhost:8002/api/pagos/123
```

Respuesta cuando SÍ tiene pago aprobado:

json

```
{
  "success": true,
  "estado": "aprobado",
  "pago": {
    "id": 456,
    "usuario_id": 123,
    "curso_id": 789,
    "estado": "aprobado",
    "monto": 99.99,
    "metodo": "paypal"
  }
}
```

Respuesta cuando NO tiene pago:

```
json
```

```
{  
  "success": true,  
  "estado": "no_pagado",  
  "message": "No se encontró pago para este curso"  
}
```

Obtener historial de pagos de usuario:

```
bash
```

```
curl -H "Authorization: Bearer <token>" \  
http://localhost:8002/api/pagos/user/history
```

Comunicación con Otros Servicios

El Payment Service llama a:

- **User Service (puerto 8004):** GET /api/users/:id
- **Course Service (puerto 8003):** GET /api/courses/:id

Variables de entorno para servicios externos:

```
bash
```

```
USER_SERVICE_URL=http://localhost:8004  
COURSE_SERVICE_URL=http://localhost:8003
```

Variables de Entorno Necesarias

bash

Database

DB_HOST=localhost

DB_PORT=3306

DB_USER=payment_user

DB_PASSWORD=payment_password

DB_NAME=payment_db

Server

PORT=8002

APP_ENV=development

JWT_SECRET=mi_clave_secreta_muy_segura

External Services URLs

USER_SERVICE_URL=http://localhost:8004

COURSE_SERVICE_URL=http://localhost:8003

PAYMENT_SERVICE_URL=http://localhost:8002

FRONTEND_URL=http://localhost:3000

BASE_URL=http://localhost:8002

PayPal Configuration

PAYPAL_CLIENT_ID=ASYN839bjb4gjMr6nRCc-7YYR8HutdM48kFMWhq-Sxp-PgB5c5R38yGiLBEPw

PAYPAL_SECRET=EHGs6eflLFMvOWTrhWjCWmwBmbXIL8he0dM6blbcVDFhwGStuz3PGFp_nreODGiJueoNyjxfZG1h

PAYPAL_ENV=sandbox

Coinbase Configuration

COINBASE_COMMERCE_API_KEY=your_coinbase_api_key

Arquitectura MVC

```

payment-service/
├── main.go           # Entry point
├── config/config.go   # Configuration
├── models/
│   ├── payment.go     # Payment model + requests/responses
│   └── payment_method.go # Payment methods
├── controllers/
│   ├── payment_controller.go # Main payment endpoints
│   └── webhook_controller.go # Webhook endpoints
├── services/
│   ├── payment_service.go    # Business logic
│   ├── paypal_service.go     # PayPal integration
│   ├── coinbase_service.go   # Coinbase integration
│   └── external_client.go    # External service calls
├── routes/routes.go        # Route definitions
├── utils/
│   ├── database.go          # DB connection + migrations
│   ├── response.go          # Response helpers
│   └── errors.go            # Error handling + auth middleware
├── Dockerfile
├── docker-compose.yml
└── .env

```

Docker Configuration

Para levantar el servicio:

```

bash

docker-compose up -d

```

Ports exposed:

- **Payment Service:** 8002
- **Payment DB:** 3307 (para evitar conflictos)

Casos de Uso Típicos para Microservicios Dependientes

Progress Service:

- Escuchar cuando payment.estado = "aprobado"
- Crear progreso inicial para usuario en curso
- Verificar acceso antes de permitir avance

Notification Service:

- Escuchar pagos completados → enviar email confirmación
- Escuchar pagos fallidos → enviar email retry
- Templates de notificaciones por método de pago

Analytics Service:

- Consultar historial completo de pagos
- Generar reportes de ingresos por período
- Métricas de conversión por método de pago

Admin Service:

- Gestionar reembolsos → llamar PUT /api/pagos/:id/status
- Dashboard de ventas → consultar datos agregados
- Manejo de pagos problemáticos

NOTA: Esta información es todo lo necesario para crear cualquier microservicio que dependa del Payment Service. Incluye APIs, formatos de datos, autenticación, y ejemplos de integración.